

Arabian Academy — Prove It

المهمة اليومية — اليوم الخامس: AI vs Human — سيناريو اختبار اختراق الويب

المهمة

الهدف من هذه المهمة إثبات القدرة على استخدام أداة ذكاء اصطناعي (Claude) كمساعد في كتابة سكربت بايثون لفحص أساسي في اختبار اختراق الويب، مع الاحتفاظ بعقلية أمنية نقدية (Security Mindset) والتدقيق في مخرجات الـ AI بدل نسخها ولصقها مباشرة.

السكربت المختار: أداة بسيطة لفحص وجود أهم HTTP Security Headers في استجابة موقع ويب، وهي من أساسيات مرحلة الاستطلاع (Reconnaissance) في أي اختبار اختراق ويب مصرّح به.

(1) الـ Prompt الأساسي المستخدم مع الـ AI

ي:لأب جوفي requests مكتك ماختساب طيسب نوثياب تبركد ي.ل بتكا
ندحم بجو عقوم HTTP GET لسري (1)
Security Headers لا مأ دوجو نم قحتي (2)
X-Frame-Options و Content-Security-Policy (ي) تباحتسلا ي في
Strict-Transport-Security و X-Content-Type-Options).
مصفانلا زرديهلاو قدوجوملا زرديهلا حضوي طيسب ريرقت عبطي (3)
ندليز ديفعت ريغ نم،نكمم لكش طسباب مزيا

(2) استجابة الـ AI

```
import requests

url = "http://testphp.vulnweb.com"

headers_to_check = [
    "X-Frame-Options",
    "Content-Security-Policy",
    "Strict-Transport-Security",
    "X-Content-Type-Options"
]

response = requests.get(url, verify=False)

print("Security Headers Report for:", url)
for header in headers_to_check:
    if header in response.headers:
        print(f"[FOUND] {header}: {response.headers[header]}")
    else:
        print(f"[MISSING] {header}")
```

(3) تحليلي النقدي للإجابة

الإيجابيات

- الفكرة العامة صحيحة: استخدام requests.get وفحص وجود الهيدرز عبر response.headers منطق سليم.
- قائمة الهيدرز المبدئية (X-Frame-Options, CSP, HSTS, X-Content-Type-Options) هي فعلاً من أهم عناصر الحماية.

المشاكل والثغرات التي تركها الـ AI

- verify=False: يعطل التحقق من شهادة SSL بالكامل، وهذا خطأ أمني جسيم في أداة أمنية — يفتح الباب لهجمات Man-in-the-Middle ويُخفي تنبيهات شهادة مزورة.
- عدم وجود timeout: لو الموقع بطيء أو مش راد، السكربت هيفضل معلق (hang) للأبد.

- غياب try/except بالكامل: أي خطأ شبكة (DNS، رفض اتصال، انقطاع) هيوقف البرنامج بـ Traceback غير مفهوم بدل رسالة واضحة.
- الرابط (URL) مكتوب داخل الكود (hardcoded)، فالسكربت غير قابل لإعادة الاستخدام على أي هدف آخر بدون تعديل الكود نفسه.
- لا يوجد أي تأكيد على إذن الفحص (Authorization) — في اختبار اختراق حقيقي، تشغيل أي أداة على نطاق بدون إذن كتابي يُعد فعلاً غير قانوني، والـ AI لم يضع أي حماية أو تنبيه بهذا الخصوص.
- القائمة ناقصة: لا تشمل هيدرز مهمة مثل Referrer-Policy و Permissions-Policy و X-XSS-Protection.
- لا يتعامل مع الـ Redirects، فقد يفحص المستخدم صفحة مختلفة عن الهدف الفعلي دون أن يعرف.
- لا يوجد أي حفظ للنتائج (تقرير) — الإخراج يُطبع فقط ويضيع بعد إغلاق الطرفية.
- لا توجد docstrings أو تعليقات توضيحية، ما يصعب المراجعة أو الصيانة لاحقاً.

4) التعديلات والتحسينات التي قمت بها بنفسي

- إبقاء verify=True دائماً لضمان التحقق من صحة شهادة SSL، مع رفع رسالة خطأ واضحة (SSLError) بدل السكوت عن المشكلة.
- إضافة timeout=10 ثانية (قابل للتغيير عبر --timeout) لمنع تعليق السكربت.
- تغليف الطلب بالكامل في try/except يفرّق بين SSLError و ConnectionError و Timeout و RequestException العامة، مع تسجيل الأخطاء عبر logging.
- استخدام argparse لأخذ الرابط ومعطيات أخرى من سطر الأوامر بدل تثبيته داخل الكود.
- إضافة دالة validate_url للتأكد من أن الرابط يحتوي على scheme صحيح (http/https) قبل إرسال أي طلب.
- إضافة تأكيد إلزامي (yes/no) من المستخدم يفيد أنه مصرّح له رسمياً بفحص الهدف، قبل تنفيذ أي طلب فعلي.
- توسيع قائمة الهيدرز لتشمل Referrer-Policy و Permissions-Policy و X-XSS-Protection.
- عرض الرابط النهائي (response.url) في حال حدوث Redirect، لتوضيح الصفحة التي تم فحصها فعلياً.
- إضافة دالة save_report لحفظ التقرير كملف JSON موثّق بالوقت (timestamp)، بدل الاكتفاء بالطباعة.
- إضافة docstrings وتعليقات ونظام logging منظم لتسهيل المراجعة والصيانة.

5) الكود النهائي بعد التدخل والمراجعة البشرية

```
#!/usr/bin/env python3
"""
security_headers_scanner.py

.بجو عقوم تباجتسا في HTTP Security Headers مها دوجو صخف تطيسب تميلعت ةادأ
اهرابنخاب حيرصد يباتك نذا لى لصاد و اهكلمت عقاوم لى لى طقف مبختست
.ينوناقة ريغ نوكتو دق رخا مابختسا ي (Authorized Penetration Testing).
"""

import argparse
import json
import logging
import sys
from datetime import datetime
from urllib.parse import urlparse

import requests

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
)
logger = logging.getLogger(__name__)
```

```

        SECURITY_HEADERS = [
            "Content-Security-Policy",
            "Strict-Transport-Security",
            "X-Frame-Options",
            "X-Content-Type-Options",
            "Referrer-Policy",
            "Permissions-Policy",
            "X-XSS-Protection",
        ]

        DEFAULT_TIMEOUT = 10 # يثاوث

        def validate_url(url: str) -> str:
            """ج يثاوث (http/https)."""
            parsed = urlparse(url)
            if parsed.scheme not in ("http", "https"):
                raise ValueError("مزل طبارلا http:// و https://")
            if not parsed.netloc:
                raise ValueError("مزل ريغ طبارلا")
            return url

        def scan_headers(url: str, timeout: int = DEFAULT_TIMEOUT) -> dict:
            """مجيئتلاب dict مجريو Security Headers لا دوجو صخفيو طبارلا GET لسري"""
            result = {
                "url": url,
                "timestamp": datetime.utcnow().isoformat() + "Z",
                "final_url": None,
                "status_code": None,
                "present_headers": {},
                "missing_headers": [],
                "error": None,
            }

            try:
                response = requests.get(
                    url,
                    timeout=timeout,
                    allow_redirects=True,
                    verify=True, # قتلش نم قحتلا
                    headers={"User-Agent": "SecurityHeadersScanner/1.0 (Educational)"},
                )
            except requests.exceptions.SSLError as exc:
                result["error"] = f"قتلش SSL: {exc}"
                logger.error(result["error"])
                return result
            except requests.exceptions.ConnectionError as exc:
                result["error"] = f"مقوللاب لاصتلا رتعة" {exc}"
                logger.error(result["error"])
                return result
            except requests.exceptions.Timeout:
                result["error"] = f"لاصتلا قلم تهننا" (timeout).
                logger.error(result["error"])
                return result
            except requests.exceptions.RequestException as exc:
                result["error"] = f"بطللا عانتا عقوتم ريغ مزل" {exc}"
                logger.error(result["error"])
                return result

            result["final_url"] = response.url
            result["status_code"] = response.status_code

            for header in SECURITY_HEADERS:
                if header in response.headers:

```

```

        result["present_headers"][header] = response.headers[header]
    else:
        result["missing_headers"].append(header)

    return result

def print_report(result: dict) -> None:
    print("=" * 60)
    print("Security Headers ريرقت صحف")
    print(f"بولطملا طبارلا : {result['url']}")
    if result["final_url"] and result["final_url"] != result["url"]:
        print(f"ليوحتلا مءىل : {result['final_url']}")
    print("=" * 60)

    if result["error"]:
        print(f"[!] صحفلا لشف : {result['error']}")
    return

print(f"HTTP Status Code: {result['status_code']}\n")

    print("تدوجوملا زرديهلا:")
    if result["present_headers"]:
        for header, value in result["present_headers"].items():
            print(f"  [+] {header}: {value}")
        else:
            print("تدوجوم يئما رديه يآ دجوي لا")

    print("\n(تلمتمح فعض طاقن) تصقانلا زرديهلا:")
    if result["missing_headers"]:
        for header in result["missing_headers"]:
            print(f"  [-] {header}")
        else:
            print("تدوجوم تيساسلا زرديهلا لك")

def save_report(result: dict, path: str) -> None:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(result, f, ensure_ascii=False, indent=2)
    logger.info(f"ي ريرقتلا ظفد مءىل: {path}")

def main() -> None:
    parser = argparse.ArgumentParser(
        description="Security Headers صحفلا تيميلعت ةادأ"
    )
    parser.add_argument("url", help="مصحف دارملا عقوملا طبار (لائم: https://example.com)")
    parser.add_argument("-o", "--output", help="فلم راسم JSON ريرقتلا ظفح",
        default=None)
    parser.add_argument("-t", "--timeout", type=int, default=DEFAULT_TIMEOUT, help="تلمهم"
        "يئاوتلاب لاصتلا")
    args = parser.parse_args()

    print(
        "\nنذا يلع لصاد وأ اهلكمت عقاوم يلع مادختسلا تصصخم ةادأ مءىل: يمينت"
        "\nطقف اهرابتخاب حيرص يباتك"
    )
    confirm = input("عقوملا اذه صحفلا كيمسر لك حرصم تنأ له؟ (yes/no): ").strip().lower()
    if confirm not in ("yes", "y"):
        print("نذا نودب عقاوم صحف زوجي لا. صحفلا فساقيلا مءىل.")
        sys.exit(1)

    try:
        url = validate_url(args.url)

```

```

except ValueError as exc:
    logger.error(str(exc))
    sys.exit(1)

result = scan_headers(url, timeout=args.timeout)
print_report(result)

if args.output:
    save_report(result, args.output)

if __name__ == "__main__":
    main()

```

مقارنة سريعة: نسخة الـ AI مقابل النسخة النهائية

نقطة	نسخة الـ AI	النسخة النهائية (بعد التدخل البشري)
التحقق من شهادة SSL	معطل (verify=False) — ثغرة MITM	مفعل دائماً (verify=True)
Timeout	غير موجود — قد يعلق السكريبت للأبد	10 ثوان قابلة للتعديل عبر --timeout
معالجة الأخطاء	لا يوجد try/except إطلاقاً	معالجة مفصلة لكل نوع استثناء (SSL, ...Connection, Timeout)
مصدر الرابط	مكتوب داخل الكود (hardcoded)	يُمرَّر كوسيلة سطر أوامر عبر argparse
إذن الفحص (Authorization)	غير موجود إطلاقاً	تأكيد إلزامي من المستخدم قبل بدء أي فحص
قائمة الهيدرز	4 هيدرز فقط	7 هيدرز تغطي أهم عناصر الحماية الحديثة
التعامل مع Redirects	غير مذكور، قد يفحص صفحة غير مقصودة	يوضح الرابط النهائي بعد أي تحويل
حفظ النتائج	طباعة فقط، بدون توثيق	خيار حفظ تقرير JSON مع timestamp
التوثيق والتعليقات	بدون docstrings أو تعليقات	docstrings + تعليقات + logging منظم